

To talentador 15/3/2024

1) VOF

(a) La implementación circular se tiene sentido en memoria estática

(b) EL & AVIS

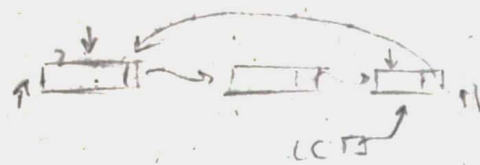
2) Lista Circular

Letra inicial ordenado

Cont Inscriptos

Area de Alumnos

La nombre y apellido AVIS (??)



i) dado un arch inscriptos20210312.TXT, que busquen ya en cada linea si no existe la letra inicial no existe insertarlo

ii) dado un alumno eliminarlo de la lista correspondiente, si queda vacia eliminar el nodo de la lista

iii) destruir el nro de lista estatica ya los operadores local & vaciar

1) void Actualizar (TLista *LC)

FILE *arch

TLista nuevo, ant, act;

char NA[24];

TElemento *x;

if (LC == fopen("inscriptos20210312.TXT", "rt")) != NULL)

while (fscanf(arch, "%s", NA) == 1)

if (LC == NULL && NA[0] != '\0') (*LC) = inicial;

strcpy(x, NA); (*LC) -> ContInscriptos += 1;

printf("%s", (*LC) -> Alumnos, x);

else if ((*LC) == NULL && NA[0] != '\0') (*LC) -> sig = inicial;

strcpy(x, NA); printf("%s", (*LC) -> sig -> Alumnos, x);

(*LC) -> sig -> ContInscriptos += 1;

else { lo busco en el resto de la lista

nuevo = (TLista) malloc(sizeof(TLista));

nuevo -> inicial = NA[0];

nuevo -> ContInscriptos;

inicialP(&nuevo -> Alumnos); printf("%s", nuevo -> Alumnos, NA);

if ((*LC) == NULL)

nuevo -> sig = nuevo;

(*LC) = nuevo;

else

if (nuevo -> inicial > (*LC) -> inicial)

nuevo -> sig = (*LC) -> sig;

(*LC) -> sig = nuevo;

(*LC) = nuevo;

casos

local al pri ✓

local al ver ✓

Busco

- lista NULL ✓
- Mayor al ver ✓
- menor al pri ✓
- Mayor al ver ✓
- Mayor al ver ✓
- Mayor al ver ✓
- Mayor al ver ✓

else

```
if (nuevo->inicial < (*LC)->sig->inicial){
```

```
nuevo->sig = (*LC)->sig;
```

```
(*LC)->sig = nuevo;
```

```
} else { // lo tengo que buscar
```

```
act = (*LC);
```

```
do {
```

```
ant = act;
```

```
act = act->sig;
```

```
} while (act != LC && act->inicial < NA[0]);
```

```
if (act->inicial == NA[0]) { // lo encontré
```

```
poner(act->Alumnos, NA);
```

```
act->contInscritos++; free(nuevo);
```

```
} else { // inserto un nuevo nodo
```

```
nuevo->sig = act;
```

```
ant->sig = nuevo;
```

```
fclose(arch);
```

```
void elimina (Tlistac *LC, char NA[5]) {
```

```
Tlistac act, ant;
```

```
TElemento x; int encontrado;
```

```
TLista aux; inicializa(aux);
```

```
if ((*LC) != NULL) {
```

```
    act = (*LC);
```

```
    do {
```

```
        ant = act;
```

```
        act = act->sig;
```

```
    } while (act != (*LC) && NA[0] > act->inicial);
```

```
    encontrado = 0;
```

```
    while (!vacio(act->Alumnos) && !encontrado) {
```

```
        sacar(act->Alumnos, &x);
```

```
        if (strcmp(x, NA) == 0)
```

```
            encontrado = 1;
```

```
        else
```

```
            poner(aux, x);
```

```
    } if (vacio(act->Alumnos)) { // debe eliminar el nodo de la lista
```

```
        if (act == (*LC)) {
```

```
            ant->sig = (*LC)->sig;
```

```
            (*LC) = ant;
```

```
        } else
```

```
            if (act == (*LC)->sig)
```

```
                (*LC)->sig = act->sig;
```

```
            else
```

```
                free(act); ant->sig = act->sig;
```

3) Se tienen $A \cap B$ y $A \cup B$ (con elementos repetidos) y un Arbol Negro AN (antes de int) desarrollar una solución que obtenga el porcentaje de los elementos de AN que tienen grado impar están en un nivel menor a k (dato), $A \cap B$

```
void Percentile (Card N AN, arbel B AB, int *Cont, int *sum, int k) {
```

Poucas e:

Posición P:

Intsi,

$$1F(1, \text{Nulo}(P))$$
$$C = \text{HIDMUS17S} (P, \underline{AN});$$
$$S = 0.$$

```
while (!null(c))
```

 $g + +,$

percentage (AN, C, AB, Cant, sum, K)

$$C = \text{Kern}(\text{injDer}(C, AN));$$
$$1f(g, y, z) \neq 0 \text{ \& Estu}(A, \text{Info}(P, a), K, 1)$$
$$(\cos n\pi) + i$$
$$(\ast_{\text{sum}}) + = \text{info}(-p, \text{B}, N);$$

```
int Esta(arbolB AB, int x, int k,
        int nact)
```

$$I_C(IAB)$$
$$1. \quad |f(w_0 + z'k)|$$

1: IF (AG → Data == X)

return 1

else

```
return Esta(A, B, K, K, nact+1) ||
```

$$\text{Estu}(AB \rightarrow \text{der}, x, k, \text{next} + 1);$$

else

return 0;

RLSE

Return 0.

7) Dado un grafo con aristas ponderadas implementado en una matriz de adyacencia (el triángulo superior), desarrollar una función int que retorne el vértice de grado k (dato de entrada) con menor costo total en las aristas que lleguen al (superior único).

int srodok (int A[][MAX], int N, int k)

Int' di costo: s, minor, minorcosto;

$$\text{new_cost} = 0; \text{new_cost}_0 = 0;$$

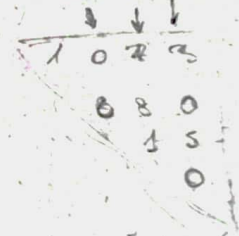
FOR ($i = 0; i < N, i++$)

$$\cos t = 0 \Rightarrow t = \frac{\pi}{2} \text{ or } \frac{3\pi}{2}$$
$$\text{for } (j=i; j \leq N; j++) \{$$
$$S_+ = A[i][j]! = 0$$
$$\omega_{\text{max}} = A[\omega]_{\text{max}}$$

If $(g = \kappa, 2\ell \cos \theta \in \text{neurocard})$

$$\text{New Cost} = \text{Cost} \cdot \alpha$$
$$p_{n+1} = i + 1;$$

return new;



totalizado 6/03/23

1) Indicar V o F, justificando o esimplificando adecuadamente

(a) La implementación de una pila en memoria estática podría hacerse iniciando el primer elemento en MAX en lugar de en -1 (siendo MAX la dimensión física)

(b) Si se aplica el recorrido de profundidad sobre un grafo no conexo el mismo podría fallar al vaciarse la pila y no conseguir llegar a un vértice no visitado de otra componente conexa

(c)  Si, porque las dimensiones lógicas del vector van de 0 a MAX-1, entonces al insertar un elemento se decrementa MAX+1 quedando MAX-1, la última posición lógica

(b) Fallaría si la condición es únicamente while (!visitado[i]) no se detendría por eso ya la pila pese a haber vaciado y al consultar el vértice que está en la pila, al estar vacía daría error.

(2) Una consuetudina de RRHH para proyectos maneja una lista simplemente enlazada con sublistas con la siguiente información:

- Id persona (ordenada, no se repite, cada 10)
- Rol (0...9, 0 es sin proyecto y sin rol que ocupa, no debe ocupar +1 rol) ^{o en + del proyecto}
- Fecha de inicio (cada 8, aaaa/mm/dd)
- Sublista de proyectos (en los que participa su participación)
Empresa (se repite, ordenada, cada 10, # es extranjera), Rol, meses de trabajo (int, sin)

(a) En un archivo de texto febrero.txt, se encuentra la información de las personas (datos correctos) que participan en los proyectos en el mes de febrero. En cada línea la persona, Empresa, Rol, término (S/N) suponiendo que existe una función que calcule la diferencia de meses (CANTM)

(b) Dados un P y E, eliminar todos los proyectos asociados a la empresa E de la persona P (pueden existir). Al finalizar, indicar en cuántos proyectos de E la persona P ha conseguido su participación (no eliminar el nodo de la lista si queda vacía)

(c) Generar una lista doble que en cada nodo tenga Empresa (ordenada, cada 10) y cantP en la que figuren solo las empresas extranjeras siendo cantP la cant de personas distintas que ha contratado.

```
typedef struct nodo {
    char[7] id;
    unsigned short int Rol;
    char[8] FechaIn;
    struct nodo * sig;
    Sublista Proyectos;
}
```

```
typedef struct nodo * TLista;
```

```
typedef struct nodito {
    char[10] Empresa;
    unsigned short int Rol;
    short int Meses;
    struct nodito * sig;
}
typedef struct nodito * Sublista;
```

2) void actualiza (Lista L)

FILE * arch;

Sublista ant, act;

char [10] E;

char [7] P;

unsigned short int R;

char T;

if ((arch = fopen("FEBLEPO.TXT", "r+")) != NULL)

{ while (fscanf(arch, "%d %s %d %d %d %d", &R, E, &P, &T, &R, &T) == 4)

{ while (strcmp(L->Id, P) != 0)

{ L = L->sig;

if (T != N)

{ ant = NULL; act = L->proyectos; } // creo el nodo

while (act != NULL && strcmp(act->Empresa, E) < 0)

{ ant = act; act = act->sig; }

if (L->proyectos == NULL || strcmp(L->proyectos->Empresa, E) > 0)

{ nuevo->sig = L->proyectos;

L->proyectos = nuevo;

else

{ ant->sig = nuevo;

nuevo->sig = act;

}

}

}

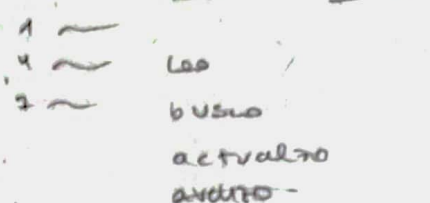
}

}

else

printf("Error al abrir el archivo");

}



nuevo = (Sublista)
malloc(sizeof(nuevo))
nuevo->Id = R
nuevo->proyectos = L->proyectos

3) void elimina (Lista L, char P[], char E[], int *conte)

Sublista act, ant;

*conte = 0;

while (L != NULL && strcmp(L->Id, P) < 0)

{ L = L->sig;

if (L != NULL && strcmp(L->Id, P) == 0)

{ ant = NULL; act = L->proyectos;

while (act != NULL && strcmp(act->Empresa, E) < 0)

{ if (strcmp(act->Empresa, E) == 0)

{ (*conte)++;

if (act == L->proyectos)

{ L->proyectos = act->sig;

free(act);

act = L->proyectos;

}

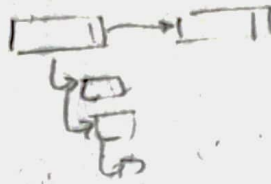
}

}

```

else {
    ant->ss = act->sis;
    Free(act);
    act = ant->sis;
}
else {
    ant = act;
    act = act->sis;
}
}
else printf("Personas inexistente");

```



```

void creaLD (Tutoria L, Tutoria *LD) {

```

```

    Sobresita act;

```

```

    nodoD * aux;

```

```

    while (L != NULL) {

```

```

        act = L->Proyectos;

```

```

        while (act != NULL) { // & act->empresa[0] = '#'

```

```

            aux = (*LD).Pri;

```

```

            while (aux != NULL && strcmp(aux->Empresa, act->Empresa) < 0)

```

```

                * aux = aux->sis;

```

```

            if (aux != NULL && strcmp == 0)

```

```

                aux->cant += 1;

```

```

            else {

```

```

                nuevo = (nodoD *) malloc (sizeof (nodoD));

```

```

                nuevo->cant = 1; nuevo->sis = NULL; nuevo->ant = NULL;

```

```

                strcpy (nuevo->Empresa, act->Empresa);

```

```

                if ((*LD).Pri == NULL) {

```

```

                    (*LD).Pri = nuevo;

```

```

                    (*LD).vet = nuevo;

```

```

                    nuevo->ant = NULL; nuevo->sis = NULL;

```

```

                }

```

```

            else

```

```

                if (strcmp (act->Empresa, (*LD).Pri->Empresa) < 0)

```

```

                    nuevo->ss = (*LD).Pri;

```

```

                    (*LD).Pri->ant = nuevo;

```

```

                    (*LD).Pri = nuevo;

```

```

                }

```

```

            else

```

```

                if (strcmp (act->Empresa, (*LD).vet->Empresa) > 0)

```

```

                    (*LD).vet->sis = nuevo;

```

```

                    nuevo->ant = (*LD).vet;

```

```

                    (*LD).vet = nuevo;

```

```

                }

```

III = III

```
else
```

```
    free(nuevo);
```

```
else
```

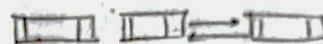
```
    nuevo → sig = aux → sig;
```

```
    aux → sig → aux = nuevo;
```

```
    nuevo → aux = aux;
```

```
    aux → sig = nuevo;
```

```
}
```



3) (Utilizar TDA N-Arbo) Dado un árbol A-Nario de enteros y un árbol binario AB que proviene de la transformación de un bosque de enteros, desarrollar una función int que retorne la cantidad de árboles del bosque que contenían una cantidad de claves que estaba en el AN en el nivel k

```
int cantidadArboles(arbol a, arbol AN, int k){
```

```
    int cant = 0, suma;
```

```
    while(a != NULL){
```

```
        suma = a → dato + sumaClaves(a → der);
```

```
        cant += Esta(AN, Raiz(AN), suma, k, 2);
```

```
        a = a → der;
```

```
    }
```

```
    return cant;
```

```
int Esta(arbol AN, posición p, int x, int k, int act){
```

```
    if(!Nula(p)){
```

```
        if(act < k)
```

```
            return Esta(AN, p → hijo1, x, k, act+1) || Esta(AN, p → hijo2, x, k, act+1);
```

```
        else
```

```
            return (info(p) == x) || Esta(AN, p → hijo1, x, k, act+1) || Esta(AN, p → hijo2, x, k, act+1);
```

```
    else
```

```
        return 0;
```

```
int sumaClaves(arbol a){
```

```
    if(a != NULL)
```

```
        return a → dato + sumaClaves(a → der) + suma(a → izq);
```

```
    else
```

```
        return 0;
```

```
}
```

④ UNITAR TDA Pila Dada una matriz de adyacencia que representa un disgrafo acíclico de N vértices (numerados del 1 a N) con aristas ponderadas y una pila P que contiene en cada elemento v (un vértice 1 a N , ordenada) y un G , determinar recursivamente sobre el grafo una de las estructuras si todos los v tienen grado de salida G . Definir el tipo de pila estática y desarrollar las funciones unitarias

```

int verifica(TPila P, int G){
    TElemento *x;
    if (!vacía(P)) {
        sacaP(&P, &x);
        if (x.grado != G)
            return 0;
        else
            return verifica(P, G);
    }
    else
        return 1;
}

```

tipo pila estática

```

typedef struct {
    TElemento datos[MAX];
    int tope;
} TPila;

```

```

int vacía(TPila P){
    return P.tope == -1;
}

```

```

void sacaP(TPila *P, TElemento *x){
    if ((*P).tope != -1)
        *x = (*P).datos[(*P).tope - 1];
}

```